SQL en Oracle Database

DDL y DCL: Guía Técnica y Práctica

© Agustín Crespo Valencia

Apuntes de Clase

Versión: Oracle 19c / 21c / 23c

Índice

1. Conceptos Fundamentales **1**

2. Acceso y Conexión mediante SQL*Plus **2**
2.1. Sintaxis Genérica de Conexión 2
2.2. Acceso como Administrador (SYSDBA) 2
2.3. Acceso como Usuario Común 3
2.4. Navegación y Verificación del Contexto 3
2.4.1. Verificar ubicación (SHOW CON_NAME) 3
2.4.2. Cambiarse de PDB (ALTER SESSION) 4

3. Arquitectura y Entorno de Trabajo **5**
3.1. Concepto de Arquitectura Multitenant (CDB vs PDB) 5
3.2. Creación de una Nueva PDB en Linux 5
3.2.1. Estrategia de Creación 5
3.2.2. Ejemplo Práctico Paso a Paso 6
3.2.3. Ciclo de Vida y Estados de una PDB 6
3.2.4. Instrucciones de Gestión y Operación 7

4. Tipos de Datos en Oracle **9**
4.1. Cadenas de Caracteres: El caso VARCHAR2 9
4.2. Charsets y Configuración Regional (NLS) 9
4.3. Numéricos: NUMBER y sus Alias 10
4.4. Fechas y Horas: DATE y TIMESTAMP 11
4.4.1. Tipos de Datos Temporales 11
4.4.2. Configuración Regional (NLS) y Formato de Entrada 11
4.4.3. Conversión Explícita (TO_DATE y TO_TIMESTAMP) 12
4.4.4. La Fecha Actual (SYSDATE) 12

5. Tipos de Datos Avanzados **13**
5.1. Objetos Grandes (LOBs) 13
5.2. Tipos Semiestructurados 13
5.3. El Tipo Booleano (BOOLEAN) 13
5.3.1. En versiones Oracle 23c y superiores 13
5.3.2. En versiones anteriores (19c, 21c) 14

6. DCL I: Gestión de usuarios y Seguridad básica **15**
6.1. Fundamentos de Almacenamiento: Tablespaces 15
6.1.1. Tablespaces por Defecto 15
6.1.2. Crecimiento de los Tablespaces (AUTOEXTEND) 15
6.2. Creación de Usuarios 16
6.2.1. Ejemplo 1: Usuario Básico 16
6.2.2. Ejemplo 2: Usuario con Tablespace Personalizado 16
6.3. Gestión de Privilegios: GRANT y REVOKE 16
6.3.1. Conceder Permisos (GRANT) 16
6.3.2. Retirar Permisos (REVOKE) 17
6.4. Aplicación de Cambios: ¿Es necesario Flush Privileges? 17
6.5. Borrado de Usuarios (DROP) 17

7. DDL I: Creación de Tablas y Restricciones **18**
7.1. Reglas de Nombramiento y Convenciones . . . 18
7.2. Tipos de Restricciones (Constraints) . . . 18
7.3. Creación de Tablas: Sintaxis y Ejemplos . . . 18
7.3.1. Caso 1: Tabla Maestra sin Constraints Nombrados . . . 19
7.3.2. Caso 2: Tabla Maestra con Constraints Nombrados (Recomendado) . . . 19
7.3.3. Caso 3: Tabla Detalle con Claves Foráneas . . . 19
7.4. Otros Objetos DDL Comunes . . . 20

8. DDL II: Secuencias y Columnas de Identidad **21**
8.1. Secuencias (SEQUENCES) . . . 21
8.2. Columnas de Identidad (IDENTITY) . . . 22

9. DDL III: Vistas (VIEWS) **23**
9.1. Utilidad y Ventajas . . . 23

10. DDL IV: Sinónimos (SYNONYMS) **24**
10.1. Tipos y Casos de Uso . . . 24
10.2. Consulta y Diccionario de Datos . . . 24
10.3. Permisos Necesarios (DCL) . . . 25

11. DDL V: Modificación y Borrado de Objetos **26**
11.1. Modificación de Estructura (ALTER TABLE) . . . 26
11.1.1. Añadir Columnas (ADD) . . . 26
11.1.2. Modificar Columnas (MODIFY) . . . 26
11.1.3. Renombrar Columnas (RENAME COLUMN) . . . 26
11.1.4. Eliminar Columnas (DROP vs SET UNUSED) . . . 27
11.2. Gestión de Restricciones (Enable/Disable) . . . 27

12. Gestión de la papelera y formas de borrado **28**
12.1. Eliminación de Tablas: DROP vs TRUNCATE . . . 28
12.2. La Papelera de Reciclaje (Recycle Bin) . . . 28

13. DCL II: Gestión Avanzada de Permisos **30**
13.1. Tipos de Privilegios . . . 30
13.2. La Cláusula WITH GRANT OPTION . . . 30
13.3. Tabla de Privilegios de Sistema Esenciales . . . 30
13.4. Roles Predefinidos: CONNECT y RESOURCE . . . 31
13.5. Revocación y Efecto Cascada . . . 31

14. Bibliografía y Recursos Recomendados **32**

© Agustín Crespo Valencia

1. Conceptos Fundamentales

Oracle Database utiliza una implementación potente del estándar SQL. Es vital recordar que los comandos se clasifican según su comportamiento transaccional:

- **DDL (Data Definition Language):** Define la estructura (CREATE, ALTER, DROP). **Nota crítica en Oracle:** Todos los comandos DDL ejecutan un `COMMIT` implícito. Si creas una tabla en mitad de una transacción, los cambios pendientes se guardan automáticamente y no podrás hacer `ROLLBACK`.
- **DML (Data Manipulation Language):** Gestiona datos (SELECT, INSERT, UPDATE, DELETE). Requieren `COMMIT` explícito para persistir.
- **DCL y TCL:** Control de permisos y transacciones.

2. Acceso y Conexión mediante SQL*Plus

Una vez creada la base de datos o el usuario, necesitamos conectarnos. La herramienta estándar de línea de comandos es `sqlplus`. La cadena de conexión varía drásticamente dependiendo de si somos administradores en el servidor local o usuarios conectándonos por red.

2.1. Sintaxis Genérica de Conexión

El comando `sqlplus` acepta una cadena de conexión que combina credenciales y, opcionalmente, la dirección de red del servicio (Easy Connect). Es fundamental entender la diferencia entre los dos métodos de acceso:

SINTAXIS GENÉRICA SQL*PLUS

```
sqlplus [ ⟨opciones⟩ ] [ ⟨logon⟩ ]

-- Donde ⟨logon⟩ se desglosa en:
{ ⟨user⟩[/⟨password⟩][@⟨host⟩:⟨port⟩/⟨service⟩] | / } [ AS SYSDBA ]
```

- **/ (Barra - OS Authentication):** Indica **Autenticación por Sistema Operativo**.
  - **Mecanismo:** Oracle confía en el usuario de Linux que ejecuta el comando (normalmente el usuario `oracle`) porque pertenece al grupo privilegiado del sistema (`dba`).
  - **Vía:** No usa la red (TCP/IP), sino comunicación interna entre procesos (IPC). Por ello, **no pasa por el Listener** y conecta generalmente al contenedor raíz (`CDB$ROOT`).
- **@ (Arroba - Network/Listener):** Indica una **Conexión por Red**.
  - **Mecanismo:** Obliga a la petición a salir a la tarjeta de red y pasar por el **Listener** de Oracle en el puerto especificado.
  - **Uso:** Es obligatorio para conectar a usuarios normales o para dirigirnos a una PDB específica (`freepdb1`) desde fuera. Requiere validar siempre usuario y contraseña.
- **AS SYSDBA:** Rol obligatorio para tareas administrativas de alto nivel (parar/arrancar BD).

2.2. Acceso como Administrador (SYSDBA)

Suponiendo que estamos trabajando **en la propia máquina** (o dentro del contenedor Docker), utilizamos la **Autenticación del Sistema Operativo**. Oracle confía en el usuario de Linux (normalmente `oracle`) y no nos pide contraseña de base de datos.

Código 1: Conexión de Administrador (Local y Red)

```
-- CASO 1: Conexión Local (OS Auth)
-- Usamos la barra (/). No requiere contraseña ni IP.
-- Conecta vía IPC directamente al contenedor raíz (CDB$ROOT).
sqlplus / as sysdba

-- CASO 2: Conexión POR REd a un PDB concreto (en este caso, FREEPDB1)
-- Usamos la arroba (@). Obligatorio para entrar en una PDB específica.
-- Nota: Aquí SÍ requerimos usuario (SYS), password y servicio.
sqlplus sys/Oracle_1234@localhost:1521/FREEPDB1 as sysdba
```

2.3. Acceso como Usuario Común

Los usuarios normales (como `gerente`) no tienen permiso para usar la autenticación del S.O. ni el rol `SYSDBA`. Deben conectarse siempre a través de la red (listener), especificando la **PDB** a la que pertenecen.

Código 2: Conexión de Usuario Común (Easy Connect)

```
-- OPCIÓN A: Conexión en una línea (Rápida pero insegura)
-- La contraseña queda guardada en el historial de la terminal.
sqlplus gerente/ClaveGerente@localhost:1521/freepdb1

-- OPCIÓN B: Conexión Interactiva (Recomendada)
-- Omitimos la contraseña en el comando. Oracle nos la pedirá de forma
   oculta.
sqlplus gerente@localhost:1521/freepdb1
```

Desglose de la cadena de conexión:

- `localhost`: La IP o nombre del servidor (en nuestra red local o remota).
- `1521`: El puerto estándar de Oracle, u otro si lo tiene cambiado.
- `freepdb1`: El nombre del servicio (`Service Name`) de nuestra PDB. **Es vital indicarlo expresamente:** si no se hace, se intentará conectar al CDB raíz, donde el usuario `gerente` no existe.

2.4. Navegación y Verificación del Contexto

Es muy común, especialmente al trabajar con usuarios administradores, conectar accidentalmente al contenedor raíz (`CDB$ROOT`) cuando en realidad queríamos trabajar en nuestra PDB.

Para evitar crear tablas o usuarios en el lugar equivocado (el "pasillo del edificio" en lugar de la "oficina"), debemos poder verificar nuestra ubicación y cambiarnos sin cerrar la sesión.

2.4.1. Verificar ubicación (SHOW CON_NAME)

Antes de ejecutar cualquier comando DDL (CREATE, DROP...), es una buena práctica confirmar en qué contenedor estamos.

Código 3: Verificación del Contenedor Actual

```
-- Comando de SQL*Plus para mostrar el contenedor activo
SHOW CON_NAME;

/* SALIDA TÍPICA SI ESTAMOS EN LA RAÍZ:

CON_NAME
------------------------------
CDB$ROOT
*/
```

Si la salida es `CDB$ROOT`, **no debemos** crear objetos de aplicación ahí.

2.4.2. Cambiarse de PDB (ALTER SESSION)

Si hemos entrado en la raíz por error, o necesitamos cambiar de una PDB a otra, no hace falta desconectarse y volver a entrar. Podemos cambiar el "foco" de la sesión con la siguiente instrucción.

Código 4: Cambio de Contenedor en Caliente

```
-- Nos movemos a la PDB 'freepdb1' (o la que necesitemos)
ALTER SESSION SET CONTAINER = freepdb1;

-- Es vital verificar que el cambio se ha realizado
SHOW CON_NAME;

/* SALIDA ESPERADA AHORA:

CON_NAME
------------------------------
FREEPDB1
*/
```

Requisito de Privilegios: Ten en cuenta que para "saltar" entre contenedores necesitas tener los permisos adecuados (privilegio `SET CONTAINER`). Normalmente esto lo realizan los administradores (`SYS`, `SYSTEM`), no usuarios comunes. Un usuario local creado dentro de una PDB generalmente no puede salir de ella (por lo general).

3. Arquitectura y Entorno de Trabajo

Oracle Database opera bajo una arquitectura **Multitenant** (Multinquilino), lo cual cambia radicalmente la forma de gestionar usuarios y tablas respecto a otras bases de datos como MySQL o SQL Server.

3.1. Concepto de Arquitectura Multitenant (CDB vs PDB)

Imagina la base de datos como un **edificio de oficinas**:

1. **CDB (Container Database) / CDB$ROOT:** Es el edificio, la estructura base. Gestiona la memoria, los procesos del sistema operativo y la seguridad global.
   - **No debemos trabajar aquí.** Solo se usa para tareas de administración global (el conserje del edificio).
   - Los usuarios creados aquí deben empezar por `C##` (Usuarios Comunes).
2. **PDB (Pluggable Database):** Son las oficinas alquiladas dentro del edificio.
   - **A los PDB también se les llama servicios**.
   - Aquí es donde creamos nuestros objetos (tablas, datos, usuarios normales).
   - Es un entorno aislado. Si un PDB (una oficina) se cierra, el SGBD (edificio que contiene la oficina) sigue funcionando. En las versiones *Free/XE*, suele venir una por defecto llamada `FREEPDB1`.

3.2. Creación de una Nueva PDB en Linux

Por lo general, la PDB (`FREEPDB1`) se utiliza para pruebas, pero en un entorno real (o de clase) querremos crear nuestra propia base de datos conectable aislada (como es `PDB_CLASE`).

Prerrequisitos:
Debemos estar conectados como `SYS` (Administrador) en el contenedor raíz (`CDB$ROOT`).

3.2.1. Estrategia de Creación

Para poder hacer nuestro propio PDB, necesitamos tener en cuenta que debemos preparar previamente la carpeta donde se alojará nuestro nuevo `PDB` desde el sistema operativo, creando primero el directorio de datos dentro de `oradata`, donde se almacenarán todos los datos.

```
mkdir -p /opt/oracle/oradata/FREE/pdb_clase/
# Asegúrate: usuario y grupo 'oracle' tiene permisos +rw sobre esa carpeta
```

Usaremos la base de datos semilla (`PDB$SEED`) como plantilla. En Linux, es vital gestionar bien las rutas de los ficheros físicos (`.dbf`). Usaremos `FILE_NAME_CONVERT` para clonar la estructura de archivos de la semilla a una nueva carpeta.

SINTAXIS GENÉRICA

```
CREATE PLUGGABLE DATABASE ⟨nombre_pdb⟩
    ADMIN USER ⟨admin_local⟩ IDENTIFIED BY ⟨password⟩
    FILE_NAME_CONVERT = (
        '⟨ruta_origen_seed⟩',
        '⟨ruta_destino_nueva⟩'
```

```
);
```

Esta instrucción copiará el contenido desde la plantilla original (PDB_SEED, por lo general) hasta la PLUGGABLE DATABASE nueva que estamos creando.

3.2.2. Ejemplo Práctico Paso a Paso

Para crear un PDB nuevo, necesitamos localizar la ruta de la plantilla, denominada PDB_SEED.

Código 5: Localización del directorio de la plantilla básica de Oracle

```
-- Buscamos la ubicación física de los ficheros de la plantilla (
   PDB$SEED)
-- Supongamos que devuelve: /opt/oracle/oradata/FREE/pdbseed/
SELECT name FROM v$datafile WHERE con_id = 2;
```

A continuación, crearemos la PLUGGABLE DATABASE nueva desde la plantilla elegida anteriormente.

Código 6: Creación de PDB en entorno Linux/Docker

```
-- Creamos la nueva PDB clonando la estructura de archivos
-- y definimos al administrador local 'admin_clase'
CREATE PLUGGABLE DATABASE pdb_clase
  ADMIN USER admin_clase IDENTIFIED BY ClaveSegura123
  FILE_NAME_CONVERT = (
    '/opt/oracle/oradata/FREE/pdbseed/',
    '/opt/oracle/oradata/FREE/pdb_clase/'
  );
```

3.2.3. Ciclo de Vida y Estados de una PDB

Una vez creada una Pluggable Database (PDB), es fundamental saber gestionar su disponibilidad.

Por defecto, cuando se reinicia el contenedor raíz (CDB), las PDBs arrancan en estado MOUNTED (cerradas), por lo que las aplicaciones no podrán conectar hasta que no se abran explícitamente.

Significado de los Estados (OPEN_MODE) El ciclo de vida de una PDB pasa por los siguientes estados principales que definen qué operaciones se pueden realizar en ella:

MOUNTED → La PDB está asociada a la instancia pero **cerrada** a los usuarios. Solo el administrador puede realizar tareas de mantenimiento estructural. Es el estado por defecto tras un reinicio.

READ WRITE → La PDB está **abierta y operativa**. Los usuarios pueden conectar, consultar y modificar datos. Es el estado normal de trabajo.

READ ONLY → La PDB está abierta, pero solo permite consultas (SELECT). No se permiten cambios (DML/DDL). Útil para bases de datos históricas o de archivo.

MIGRATE → Estado especial utilizado únicamente durante procesos de actualización (upgrade) de la base de datos.

3.2.4. Instrucciones de Gestión y Operación

A continuación, detallamos los comandos necesarios para operar con las PDBs, presentando primero la sintaxis genérica y luego ejemplos prácticos.

1. Consultar el Estado Actual Podemos verificar qué PDBs existen y su estado (`OPEN_MODE`) mediante SQL*Plus o consultando el diccionario de datos.

SINTAXIS GENÉRICA

```
-- Comando simplificado (SQL*Plus)
SHOW PDBS;

-- Consulta detallada
SELECT con_id, name, open_mode
    FROM v$pdbs;
```

Código 7: Ejemplo: Consultar PDBs y sus estados

```
-- Opción A: Comando rápido de SQL*Plus
SHOW PDBS;

-- Opción B: Consulta SQL estándar (más detallada)
SELECT con_id, name, open_mode
    FROM v$pdbs
    ORDER BY con_id;
```

2. Arrancar y Parar una PDB Para cambiar el estado de una PDB específica (o de todas), utilizamos la sentencia `ALTER PLUGGABLE DATABASE`.

SINTAXIS GENÉRICA

```
-- Cambio de estado para una PDB concreta.
ALTER PLUGGABLE DATABASE ⟨nombre_pdb⟩ { OPEN | CLOSE [IMMEDIATE] };

-- Cambio de estado para todos los PDB del SGBDR.
ALTER PLUGGABLE DATABASE ALL { OPEN | CLOSE [IMMEDIATE] };
```

Código 8: Ejemplo: Apertura y Cierre de PDBs

```
-- Abrimos la nueva base de datos para permitir conexiones
ALTER PLUGGABLE DATABASE pdb_clase OPEN;

-- Parar una PDB (Paso a MOUNTED)
-- IMMEDIATE fuerza el cierre desconectando sesiones activas
ALTER PLUGGABLE DATABASE pdb_clase CLOSE IMMEDIATE;

-- Abrir todas las PDBs del contenedor de una vez
ALTER PLUGGABLE DATABASE ALL OPEN;
```

3. Persistencia del Estado (SAVE STATE) Como Oracle no recuerda por defecto que una PDB debe estar abierta tras reiniciar el servidor, debemos guardar su estado actual explícitamente para evitar aperturas manuales tras cada reinicio.

SINTAXIS GENÉRICA

```
-- Guardar estado actual (para auto-arranque)
ALTER PLUGGABLE DATABASE ⟨nombre_pdb⟩ SAVE STATE;

-- Olvidar estado guardado
ALTER PLUGGABLE DATABASE ⟨nombre_pdb⟩ DISCARD STATE;
```

Código 9: Ejemplo: Persistencia de arranque

```
-- Abrimos la PDB (Paso previo necesario)
ALTER PLUGGABLE DATABASE pdb_clase OPEN;

-- Guardamos el estado actual para que persista tras reiniciar el SGBDR
-- Oracle recordará que debe arrancarla en el próximo reinicio
ALTER PLUGGABLE DATABASE pdb_clase SAVE STATE;

-- (Opcional) Si queremos que deje de arrancar automáticamente:
ALTER PLUGGABLE DATABASE pdb_clase DISCARD STATE;
```

4. Cambio de Contexto (Navegación) Si estamos conectados al contenedor raíz (CDB) y queremos trabajar dentro de una PDB específica sin desconectarnos, cambiamos el contenedor de la sesión.

SINTAXIS GENÉRICA

```
ALTER SESSION SET CONTAINER = ⟨nombre_pdb⟩;
```

Código 10: Ejemplo: Cambio de entorno de trabajo

```
-- Cambiamos el contexto de la sesión para entrar en la nueva oficina
ALTER SESSION SET CONTAINER = pdb_clase;
```

4. Tipos de Datos en Oracle

4.1. Cadenas de Caracteres: El caso VARCHAR2

Tipo Oracle	Alias ANSI	Descripción Técnica
CHAR(n)	CHARACTER(n)	Cadena de longitud fija. Si el texto es más corto, Oracle rellena con espacios a la derecha para completar los n CHAR que debe almacenar.
VARCHAR2(n)	VARCHAR(n)	Cadena de longitud variable. El '2' indica que sigue la lógica de Oracle donde **NULL es igual a cadena vacía**. Se recomienda usar siempre este tipo.
NVARCHAR2(n)	NCHAR VARYING(n)	Cadena variable que usa exclusivamente el conjunto de caracteres nacional (generalmente UTF-16).

Cuadro 1: Tipos de Cadenas

Dilema NULL vs Cadena Vacía: En el estándar ANSI, '' (la cadena vacía) es un valor distinto de NULL. En Oracle, '' se convierte automáticamente en NULL.

4.2. Charsets y Configuración Regional (NLS)

Oracle utiliza el sistema **NLS (National Language Support)** para definir cómo se guardan los caracteres y cómo se muestran las fechas o monedas.

Es vital conocer qué juego de caracteres tiene nuestra base de datos para evitar problemas con tildes o caracteres especiales (ñ, €, etc.).

Existen dos parámetros fundamentales que definen el almacenamiento:

- **NLS_CHARACTERSET:** Define el juego de caracteres para los tipos VARCHAR2, CHAR y CLOB. Lo ideal hoy en día es AL32UTF8 (Unicode).
- **NLS_NCHAR_CHARACTERSET:** Define el juego de caracteres para los tipos nacionales (NVARCHAR2, NCHAR, NCLOB). Suele ser AL16UTF16.

Código 11: Consulta de Parámetros NLS

```
-- Nosotros consultamos la vista del diccionario para ver la configuraci
   ón real
-- Filtramos para ver solo lo relacionado con el Character Set
SELECT parameter, value
FROM   nls_database_parameters
WHERE  parameter LIKE '%CHARACTERSET%';

/* SALIDA TIPICA:
PARAMETER                  VALUE
-------------------------- --------------------
NLS_NUMERIC_CHARACTERS     ,.        (Decimal coma, miles punto)
NLS_CHARACTERSET           AL32UTF8 (Soporta todos los idiomas)
NLS_NCHAR_CHARACTERSET     AL16UTF16
*/
```

4.3. Numéricos: NUMBER y sus Alias

La potencia de Oracle reside en el tipo `NUMBER`, capaz de almacenar desde enteros simples hasta coordenadas astronómicas con precisión exacta.

SINTAXIS GENÉRICA

```
NUMBER [ ( precisión [, escala ] ) ]
```

- **Precisión (p):** Número total de dígitos significativos (1 a 38).
- **Escala (s):** Número de dígitos a la derecha de la coma decimal (-84 a 127).

Definición	Dato Entrada	Valor Almacenado / Resultado
`NUMBER`	1234.5678	1234.5678 (Máxima precisión disponible)
`NUMBER(5)`	123.45	123 (Redondea a entero)
`NUMBER(5)`	123456	**ERROR** (Excede la precisión de 5 dígitos)
`NUMBER(5, 2)`	123.456	123.46 (Redondea a 2 decimales)
`NUMBER(5, 2)`	1234.56	**ERROR** (Necesita 4 dígitos a la izquierda + 2 decimales = 6 total)
`NUMBER(5, -2)`	12345	12300 (Redondea a las centenas)

Cuadro 2: Comportamiento de Precisión y Escala en NUMBER

ANSI / SQL (alias)	Mapeo Oracle (interno)	Uso Recomendado
`INTEGER` `INT`	`NUMBER(38)`	Enteros puros.
`SMALLINT`	`NUMBER(38)`	Enteros (sin ahorro de espacio real en Oracle).
`DECIMAL(p,s)`	`NUMBER(p,s)`	Importes financieros exactos.
`NUMERIC(p,s)`	`NUMBER(p,s)`	Sinónimo estricto de DECIMAL.
`FLOAT(n)`	`FLOAT(n)`	Cálculos científicos (precisión variable).
`REAL`	`FLOAT(63)`	Precisión simple aproximada.
`DOUBLE PRECISION`	`FLOAT(126)`	Precisión doble estándar ANSI.
Nativo (IEEE 754)	`BINARY_FLOAT`	Coma flotante de 32 bits. Más rápido que NUMBER, pero no apto para dinero.
Nativo (IEEE 754)	`BINARY_DOUBLE`	Coma flotante de 64 bits. Estándar científico moderno.

Cuadro 3: Alias Numéricos y su Conversión

4.4. Fechas y Horas: DATE y TIMESTAMP

Oracle maneja las fechas de forma interna como formatos numéricos, lo que permite realizar cálculos aritméticos (restar fechas, sumar días) con gran facilidad. Sin embargo, su visualización y entrada dependen estrictamente de la configuración regional.

4.4.1. Tipos de Datos Temporales

Tipo de Dato	Precisión	Descripción Técnica
`DATE`	Al segundo (1s)	El estándar de Oracle. Almacena: Siglo, Año, Mes, Día, Hora, Minuto y Segundo. **Nota:** Aunque hagamos un `SELECT` y solo veamos el día (como, por ejemplo, 12/05/23), la hora (00:00:00) siempre está guardada internamente.
`TIMESTAMP`	Fracciones de segundo	Extensión de `DATE` para alta precisión. Almacena hasta 9 decimales de segundo (nanosegundos). Útil para logs de auditoría o tiempos de ejecución.
`TIMESTAMP WITH TIME ZONE`	+ Zona Horaria	Igual que TIMESTAMP pero almacena el desplazamiento horario (para España es +01:00). Vital para aplicaciones globales.

Cuadro 4: Tipos de Datos de Fecha y Hora

4.4.2. Configuración Regional (NLS) y Formato de Entrada

Una confusión habitual es pensar que la fecha se guarda con formato “DD/MM/YYYY”. No es así. Oracle la guarda en binario y la **muestra** según el parámetro `NLS_DATE_FORMAT`.

Si intentamos insertar una fecha como cadena de texto (’20-01-2023’) sin usar funciones de conversión, Oracle intentará interpretarla usando su formato por defecto. Si no coinciden, recibiremos el error `ORA-01861`.

Código 12: Consulta del formato de fecha actual

```
-- Comprobamos como espera Oracle recibir las fechas en mi sesion
SELECT parameter, value
    FROM   nls_session_parameters
    WHERE  parameter IN ('NLS_DATE_FORMAT', 'NLS_TIMESTAMP_FORMAT');

/* SALIDA TIPICA EN ESPAÑA:
NLS_DATE_FORMAT        DD/MM/RR
NLS_TIMESTAMP_FORMAT   DD/MM/RR HH24:MI:SSXFF
*/
```

El formato RR vs YY: Oracle suele usar `RR` en lugar de `YY` para el año. `RR` es un formato inteligente para solucionar el “efecto 2000”:

- Si escribes ’98’ en formato RR, Oracle interpreta 1998.
- Si escribes ’15’ en formato RR, Oracle interpreta 2015.

4.4.3. Conversión Explícita (TO_DATE y TO_TIMESTAMP)

Para evitar errores y ambigüedades, **nunca** debemos confiar en el formato implícito. La buena práctica es convertir siempre las cadenas usando `TO_DATE`, especificando la máscara de formato exacta.

SINTAXIS GENÉRICA

```
TO_DATE( 'cadena_fecha', 'máscara_formato' )
TO_TIMESTAMP( 'cadena_fecha', 'máscara_formato' )
```

Máscaras comunes:

- `YYYY`: Año de 4 dígitos (2024).
- `MM`: Mes numérico (01-12). `MON`: Abreviatura (ENE/JAN).
- `HH24`: Hora en formato 24h (00-23). **Importante:** Usar `HH` implica formato 12h y requiere AM/PM.
- `MI`: Minutos (00-59). **Ojo:** No confundir con `MM` (mes).
- `SS`: Segundos.
- `FF`: Fracciones de segundo (solo para TIMESTAMP).

Código 13: Inserción de Fechas con y sin Hora

```
-- Ejemplo: Insercion simple (La hora quedara a 00:00:00)
INSERT INTO proyecto (id, f_inicio)
    VALUES (1, TO_DATE('15/08/2024', 'DD/MM/YYYY'));

-- Ejemplo: Insercion con Hora precisa (Resolucion de segundos)
-- Observad el uso de HH24 para las 14:30
INSERT INTO auditoria (usuario, fecha_acceso)
    VALUES ('ADMIN',
            TO_DATE('30-01-2024 14:30:59', 'DD-MM-YYYY HH24:MI:SS'));

-- Ejemplo: Insercion de alta precision (Milisegundos)
INSERT INTO log_sistema (evento, instante)
    VALUES ('ERROR_CRITICO',
            TO_TIMESTAMP('2024-02-28 23:59:59.123456',
                         'YYYY-MM-DD HH24:MI:SS.FF'));
```

4.4.4. La Fecha Actual (SYSDATE)

Para insertar el momento actual del servidor (muy común en campos que contengan una fecha y hora con mucha precisión, como por ejemplo`fecha_creacion`), no convertimos nada, simplemente usamos las funciones del sistema:

- `SYSDATE`**:** Devuelve fecha y hora actual del sistema operativo del servidor (tipo DATE).
- `SYSTIMESTAMP`**:** Devuelve fecha, hora y zona horaria con precisión de microsegundos.

```
-- Guardamos el momento exacto de la creacion del registro
INSERT INTO departamento (id_dep, nombre, f_creacion)
    VALUES (50, 'I+D', SYSDATE);
```

5. Tipos de Datos Avanzados

5.1. Objetos Grandes (LOBs)

Diseñados para almacenar grandes volúmenes de datos no estructurados (hasta 4GB por defecto, ampliable a TeraBytes).

Tipo LOB	Características y Uso
`CLOB`(Character Large Object)	Almacena textos enormes en el set de caracteres de la base de datos. Soporta funciones de cadena como `SUBSTR` o `INSTR`.
`NCLOB`(National CLOB)	Igual que CLOB pero utiliza el set de caracteres nacional (Unicode). Ideal para documentos multilenguaje.
`BLOB`(Binary Large Object)	Almacena datos binarios tal cual (imágenes, PDF, ejecutables). La base de datos no interpreta el contenido, solo lo guarda.
`BFILE`(Binary File)	**Solo lectura.** Almacena un puntero a un archivo en el sistema operativo del servidor. **Restricción:** No se puede escribir en el archivo desde SQL (solo leer). Si el archivo se borra en el SO, el enlace se rompe. Requiere crear un objeto `DIRECTORY`.

Cuadro 5: Tipos LOB (Large Objects)

5.2. Tipos Semiestructurados

Oracle permite almacenar y validar datos que tienen una estructura interna flexible.

- **XMLTYPE:** Tipo nativo para almacenar documentos XML.
  - Permite validar el contenido contra un esquema XML (XSD) registrado en la base de datos mediante `DBMS_XMLSCHEMA`.
  - Permite consultas potentes usando XQuery y XPath sobre el contenido.
- **JSON:** (Introducido como tipo nativo en 21c).
  - Almacena documentos JSON en un formato binario optimizado llamado **OSON**.
  - Mucho más rápido que guardarlo como texto. Permite validación de estructura.
  - **Pre-21c:** Se guardaba en `CLOB` o `VARCHAR2` añadiendo una restricción: `CONSTRAINT ck_json CHECK (columna IS JSON)`.

5.3. El Tipo Booleano (BOOLEAN)

Históricamente, Oracle SQL **no soportaba** el tipo `BOOLEAN` (solo existía en el lenguaje procedural PL/SQL). Esto ha cambiado recientemente.

5.3.1. En versiones Oracle 23c y superiores

Existe el tipo nativo `BOOLEAN`.

SINTAXIS GENÉRICA

```
activo BOOLEAN DEFAULT TRUE
```

5.3.2. En versiones anteriores (19c, 21c)

Debemos simularlo. Las estrategias más comunes son:

- **Opción Numérica:** Usar `NUMBER(1)` con restricción.

  ```
  activo NUMBER(1) CHECK (activo IN (0, 1))
  ```

- **Opción Carácter:** Usar `CHAR(1)` con 'S'/'N' o 'Y'/'N' o 'T'/'F'.

  ```
  activo CHAR(1) CHECK (activo IN ('S', 'N'))
  ```

6. DCL I: Gestión de usuarios y Seguridad básica

6.1. Fundamentos de Almacenamiento: Tablespaces

Antes de crear usuarios, debemos entender dónde se guardarán sus datos. Para organizar mejor la información, se puede usar un **Tablespace**, que es una **unidad lógica de almacenamiento** que agrupa estructuras de datos (tablas, índices, etc.).

Físicamente, un tablespace se corresponde con uno o más ficheros en el disco llamados `DATAFILES`.

6.1.1. Tablespaces por Defecto

Toda base de datos Oracle viene con un conjunto de tablespaces predefinidos, cada uno con una función vital:

- **SYSTEM:** Es el corazón de la base de datos. Contiene el **Diccionario de Datos** (metadatos sobre tablas, usuarios, vistas). Nunca debemos crear objetos de usuario aquí.
- **SYSAUX:** Auxiliar del sistema. Almacena componentes de Oracle (estadísticas, AWR, auditoría).
- **TEMP:** Usado para operaciones temporales que no caben en memoria, como ordenaciones grandes (`ORDER BY`) o uniones complejas. Su contenido se borra al reiniciar.
- **UNDOTBS1:** Almacena información de "deshacer". Permite hacer `ROLLBACK` y garantiza la consistencia de lectura.
- **USERS:** Es el tablespace estándar donde se guardan las tablas y datos de los usuarios normales.

6.1.2. Crecimiento de los Tablespaces (AUTOEXTEND)

Un tablespace se compone de Datafiles. Cuando creamos un usuario, este utiliza el espacio dentro del tablespace asignado. El crecimiento se gestiona a nivel de fichero físico:

1. **Tamaño Fijo:** Si el Datafile se llena y no tiene activado el auto-crecimiento, la base de datos lanzará el error `ORA-01653` (Unable to extend segment).
2. **Autoextend:** Configuramos el fichero para crecer automáticamente.
   - **NEXT:** Cantidad de espacio que se añade cada vez que se necesita crecer.
   - **MAXSIZE:** Límite superior total que puede alcanzar el fichero.

Código 14: Creación de Tablespace con Crecimiento Controlado

```
-- Creamos un tablespace llamado 'ts_ventas'
-- Definimos un tamaño inicial de 100MB
-- Configuramos que crezca de 50MB en 50MB (NEXT)
-- Limitamos su crecimiento máximo a 2GB para no llenar el disco (
   MAXSIZE)
CREATE TABLESPACE ts_ventas
  DATAFILE 'ventas01.dbf' SIZE 100M
  AUTOEXTEND ON NEXT 50M MAXSIZE 2G;
```

6.2. Creación de Usuarios

La creación de un usuario implica definir su autenticación y sus límites de almacenamiento (Cuotas).

SINTAXIS GENÉRICA

```
CREATE USER ⟨nombre_usuario⟩
  IDENTIFIED BY ⟨contraseña⟩
  [ DEFAULT TABLESPACE ⟨ts_datos⟩ ]
  [ TEMPORARY TABLESPACE ⟨ts_temp⟩ ]
  [ QUOTA { ⟨tamaño⟩ | UNLIMITED } ON ⟨ts_datos⟩ ];
```

6.2.1. Ejemplo 1: Usuario Básico

Código 15: Creación de Usuario Estándar

```
-- Nos conectamos como SYSTEM en la PDB
-- Al no indicarlo, utilizará los tablespace por defecto
-- Creamos al usuario y le damos espacio limitado en USERS
CREATE USER gerente
  IDENTIFIED BY ClaveGerente
  QUOTA 100M ON users;
```

6.2.2. Ejemplo 2: Usuario con Tablespace Personalizado

En este ejemplo, vinculamos al usuario con el tablespace `ts_ventas` que creamos con `NEXT` y `MAXSIZE` en el punto anterior.

Código 16: Usuario asignado a Tablespace Auto-extensible

```
-- Creamos un usuario para el departamento de ventas
-- Le asignamos el tablespace ts_ventas, que configuramos con límites
-- Al definirle un DEFAULT TABLESPACE, todos los objetos se guardan ahí
CREATE USER vendedor
  IDENTIFIED BY Ventas2K
  DEFAULT TABLESPACE ts_ventas
  QUOTA UNLIMITED ON ts_ventas;

-- Nota: Aunque la cuota sea UNLIMITED, el usuario no podrá exceder
-- los 2GB totales definidos en el MAXSIZE del tablespace ts_ventas.
```

6.3. Gestión de Privilegios: GRANT y REVOKE

6.3.1. Conceder Permisos (GRANT)

Un usuario recién creado no puede conectarse hasta recibir permisos.

Código 17: Asignación de Roles

```
-- Asignamos roles básicos para trabajar
GRANT CONNECT, RESOURCE TO gerente;

-- Asignamos un permiso específico de sistema
GRANT CREATE VIEW TO gerente;
```

6.3.2. Retirar Permisos (REVOKE)

Si necesitamos quitar privilegios a un usuario sin eliminarlo, utilizamos el comando `REVOKE`. Es fundamental para el principio de "mínimo privilegio".

SINTAXIS GENÉRICA

```
REVOKE { ⟨privilegio⟩ | ⟨rol⟩ } [, ...] FROM ⟨usuario⟩;
```

Código 18: Retirada de Permisos

```
-- Retiramos el rol RESOURCE al gerente (ya no podrá crear tablas)
REVOKE RESOURCE FROM gerente;

-- Retiramos el permiso específico de crear vistas
REVOKE CREATE VIEW FROM gerente;
```

6.4. Aplicación de Cambios: ¿Es necesario Flush Privileges?

Es común en administradores que vienen de *MySQL* o *MariaDB* preguntar por el comando `FLUSH PRIVILEGES`.

- **En Oracle Database: NO existe** ni es necesario ejecutar ningún comando de refresco.
- **Razón:** Oracle gestiona los permisos directamente en el Diccionario de Datos de forma transaccional inmediata.
- **Efecto:** En el momento exacto en que nosotros ejecutamos un `GRANT` o un `REVOKE`, el cambio es efectivo. Si el usuario intenta realizar la acción un milisegundo después, la base de datos validará el permiso actualizado instantáneamente.

6.5. Borrado de Usuarios (DROP)

Para eliminar un usuario, usamos el comando `DROP USER`. Sin embargo, existe una restricción de seguridad si el usuario posee objetos.

Si el usuario (por ejemplo, `gerente`) ha creado tablas, Oracle **bloqueará** el borrado simple. Para forzarlo, usamos la cláusula **CASCADE**.

¿Qué implica el CASCADE?

1. Primero, Oracle elimina todos los objetos (tablas, vistas, secuencias) propiedad de ese esquema.
2. Después, elimina al usuario y sus referencias de seguridad.

Código 19: Borrado Recursivo con Cascade

```
-- Eliminamos el usuario y todas sus tablas/objetos
-- Si no usamos CASCADE y el usuario tiene tablas, recibiremos un error.
DROP USER gerente CASCADE;
```

7. DDL I: Creación de Tablas y Restricciones

La tabla es la unidad básica de almacenamiento de datos. Aunque la sentencia `CREATE TABLE` parece sencilla, la clave para un buen diseño de base de datos reside en la correcta definición de las **Restricciones (Constraints)**.

7.1. Reglas de Nombramiento y Convenciones

En Oracle, los nombres de tablas y columnas deben comenzar por una letra y pueden tener hasta 128 caracteres.

Tendremos en cuenta lo siguiente:

- **Tablas:** Nombres en **singular** (como `departamento` en lugar de `departamentos`).
- **Restricciones:** Patrón `tabla_tipo-restriccion_columna` (por ej.: `empleado_pk_id`).

7.2. Tipos de Restricciones (Constraints)

Las restricciones garantizan la integridad de los datos. Es una buena práctica nombrar siempre las restricciones explícitamente para identificar errores fácilmente en el futuro.

Restricción	Función
PK **PRIMARY KEY**	Identifica de forma única cada fila. Implica `UNIQUE` y `NOT NULL`. Solo puede haber una por tabla.
FOREIGN KEY **FK**	Garantiza la integridad referencial. El valor debe existir en la tabla padre o ser `NULL`.
NOT NULL **NN**	Impide que la columna se quede vacía.
UNIQUE **UK**	Evita valores duplicados en una columna (pero permite múltiples `NULL`).
CHECK **CK**	Valida que el dato de una columna cumpla una condición lógica, como por ejemplo en (`salario >0`).

Cuadro 6: Restricciones de Integridad

7.3. Creación de Tablas: Sintaxis y Ejemplos

Existen dos formas de definir restricciones: a nivel de columna (*inline*) o a nivel de tabla (*out-of-line*).

SINTAXIS GENÉRICA

```
CREATE TABLE ⟨nombre_tabla⟩ (
    -- DEFINICION DE COLUMNAS
    columna1 ⟨tipo_dato⟩ [DEFAULT ⟨valor⟩]
        [ [CONSTRAINT ⟨nombre⟩ ] definicion_restricción ],
    columna2 ⟨tipo_dato⟩ [DEFAULT ⟨valor⟩]
        [ [CONSTRAINT ⟨nombre⟩ ] definicion_restricción ],
    columna3 ⟨tipo_dato⟩ [DEFAULT ⟨valor⟩]
        [ [CONSTRAINT ⟨nombre⟩ ] definicion_restricción ],
    columnaX ⟨tipo_dato⟩ ...,
```

```
    -- RESTRICCIONES DE TABLA (Al final, separadas por comas)
    [ [CONSTRAINT ⟨nombre_uk⟩ ] UNIQUE (⟨cols⟩) ],
    [ [CONSTRAINT ⟨nombre_ck⟩ ] CHECK (⟨condición⟩) ],
    [ [CONSTRAINT ⟨nombre_pk⟩ ]
        PRIMARY KEY ( [col | (col1, col2, ...)] ) ],
    [ [CONSTRAINT ⟨nombre_fk⟩ ]
        FOREIGN KEY ( [col | (col1, col2, ...)] )
            REFERENCES tabla_padre  ( [col | (col1, col2, ...)] ) ]
);
```

7.3.1. Caso 1: Tabla Maestra sin Constraints Nombrados

Oracle permite crear restricciones sin darles nombre. En este caso, el sistema asignará nombres aleatorios del tipo `SYS_C0012345`.

Esto es más rápido de escribir pero dificulta el mantenimiento posterior.

Código 20: Tabla Maestra con Nombres de Sistema

```
-- Creamos una tabla de regiones simple
-- Observad que indicamos PRIMARY KEY y NOT NULL sin la palabra
   CONSTRAINT
CREATE TABLE region (
    id        NUMBER(5)    PRIMARY KEY,
    nombre    VARCHAR2(50) NOT NULL,
    pais      VARCHAR2(50)
);
```

7.3.2. Caso 2: Tabla Maestra con Constraints Nombrados (Recomendado)

En este ejemplo creamos la tabla `departamento` nombrando explícitamente cada restricción siguiendo el patrón `tabla_tipo_columna`.

Esto facilita la administración y la lectura de errores.

Código 21: Tabla Maestra con Nombres Explícitos

```
-- Creamos la tabla departamento definiendo nombres claros para las
   reglas
CREATE TABLE departamento (
    id          NUMBER(5)    CONSTRAINT departamento_pk_id PRIMARY KEY,
    nombre      VARCHAR2(50) CONSTRAINT departamento_nn_nombre NOT NULL,
    ciudad      VARCHAR2(50) DEFAULT 'Madrid',
    presupuesto NUMBER(12,2) DEFAULT 0.00,

    -- Validamos que el presupuesto sea positivo usando un CHECK
    CONSTRAINT departamento_ck_presupuesto CHECK (presupuesto ≥ 0),

    -- Aseguramos nombres únicos para evitar duplicados
    CONSTRAINT departamento_uk_nombre UNIQUE (nombre)
);
```

7.3.3. Caso 3: Tabla Detalle con Claves Foráneas

Vamos a crear la tabla `proyecto` vinculada a `departamento`. Usaremos restricciones a nivel de tabla para la definición de la FK.

Código 22: Tabla Detalle (Foreign Keys)

```
-- Creamos la tabla proyecto vinculada a departamento
CREATE TABLE proyecto (
    id          NUMBER(5),
    titulo      VARCHAR2(100) CONSTRAINT proyecto_nn_titulo NOT NULL,
    f_inicio    DATE          DEFAULT SYSDATE,
    dep_id      NUMBER(5),
    estado      VARCHAR2(20)  NOT NULL,

    -- Definimos la Primary Key a nivel de tabla
    CONSTRAINT proyecto_pk_id PRIMARY KEY (id),

    -- Definimos la Foreign Key estableciendo la regla de integridad
    -- Si se borra el departamento, ponemos a NULL el campo en proyecto
    CONSTRAINT proyecto_fk_dep_id
        FOREIGN KEY (dep_id)
            REFERENCES departamento (id)
            ON UPDATE CASCADE
            ON DELETE SET NULL,

    -- Restricción de validación para el estado del proyecto
    CONSTRAINT proyecto_ck_estado
        CHECK (estado IN ('ACTIVO', 'CERRADO'))
);
```

7.4. Otros Objetos DDL Comunes

Además de tablas, el esquema de la arquitectura de una base de datos en Oracle se compone de otros objetos estructurales que es necesario conocer qué son y cómo manejarlos:

- **ÍNDICES (INDEXES):** Estructuras B-Tree (por defecto) para acelerar consultas. Oracle indexa automáticamente las `PK` y `UK`, pero las `FK` deben indexarse manualmente si se usan mucho en JOINs.
- **SECUENCIAS (SEQUENCES):** Generadores de números correlativos muy rápidos, independientes de las tablas. Aunque `IDENTITY` las reemplaza en usos simples, siguen siendo vitales en entornos de alto rendimiento.
- **VISTAS (VIEWS):** Consultas guardadas con nombre. En Oracle, pueden ser simples (solo lectura lógica) o complejas. Se gestionan con `CREATE OR REPLACE VIEW`.
- **SINÓNIMOS (SYNONYMS):** Alias para objetos. Permiten que un usuario consulte con el nombre `empleados` en lugar de `rrhh.empleados`.

8. DDL II: Secuencias y Columnas de Identidad

En el diseño de bases de datos, es fundamental contar con mecanismos que garanticen la unicidad de las claves primarias de forma eficiente. Oracle ofrece dos soluciones principales: las tradicionales `SEQUENCES` y las modernas columnas `IDENTITY`.

8.1. Secuencias (SEQUENCES)

Una secuencia es un objeto del esquema, independiente de las tablas, que genera números enteros en orden ascendente o descendente. Es altamente eficiente en entornos multiusuario porque no bloquea el acceso mientras genera el siguiente número.

SINTAXIS GENÉRICA

```
CREATE SEQUENCE ⟨nombre_secuencia⟩
  [START WITH ⟨n⟩]
  [INCREMENT BY ⟨m⟩]
  [MAXVALUE ⟨x⟩ | NOMAXVALUE]
  [CYCLE | NOCYCLE]
  [CACHE ⟨y⟩ | NOCACHE];
```

Opciones de la Secuencia Interna

Para una secuencia podemos definir el comportamiento:

- `START WITH n`: Define el valor inicial (por defecto 1).
- `INCREMENT BY n`: Define el intervalo de salto (por defecto 1).
- `CACHE n`: Mejora el rendimiento pre-generando números en memoria.
- `CYCLE`: Permite reiniciar el contador al llegar al valor máximo.

Uso de NEXTVAL y CURRVAL

Para interactuar con una secuencia se utilizan dos "pseudocolumnas" que se pueden consultar:

- `NEXTVAL`: Incrementa la secuencia y devuelve el nuevo valor.
- `CURRVAL`: Devuelve el valor actual de la secuencia en la sesión actual (requiere haber ejecutado un `NEXTVAL` previo).

Código 23: Ejemplo: Secuencia para la tabla Departamento

```
-- Secuencia que empieza en 10 y aumenta de 10 en 10, y caché de 20 nú
   meros precargados
CREATE SEQUENCE seq_dep_id
    START WITH 10
    INCREMENT BY 10
    CACHE 20;

-- Insertamos un nuevo departamento usando la secuencia
INSERT INTO departamento (id_dep, nombre, ciudad)
    VALUES (seq_dep_id.NEXTVAL, 'Marketing', 'Barcelona');
```

8.2. Columnas de Identidad (IDENTITY)

Las columnas de identidad son una mejora funcional que permite automatizar la generación de claves primarias numéricas. A diferencia de las secuencias tradicionales, la identidad está vinculada de forma intrínseca a la tabla.

Internamente, Oracle crea una secuencia "invisible" para gestionar los valores, encargándose de su mantenimiento de forma automática.

SINTAXIS GENÉRICA

```
GENERATED [ ALWAYS | BY DEFAULT [ ON NULL ] ]
AS IDENTITY [ ( opciones_de_secuencia ) ]
```

Modos de Generación

El comportamiento de la columna ante una sentencia `INSERT` depende de la opción elegida:

- `GENERATED ALWAYS`**:** Es la opción más estricta. Oracle garantiza el control total del valor. Si un usuario intenta proporcionar un valor manualmente en el `INSERT`, la base de datos lanzará un error.
- `GENERATED BY DEFAULT`**:** El sistema genera el valor solo si el usuario omite la columna. Si el usuario proporciona un valor manual, Oracle lo acepta (siempre que no viole la unicidad).
- `GENERATED BY DEFAULT ON NULL`**:** Una variante más flexible que permite que, si el usuario intenta insertar un `NULL` explícito, Oracle lo ignore y genere el siguiente número de la secuencia automáticamente.

Código 24: Ejemplos de configuración de IDENTITY

```
-- Caso 1: Control absoluto del sistema (START en 100)
CREATE TABLE hito_proyecto (
    id_hito     NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    descripcion VARCHAR2(100)
);

-- Caso 2: Permisivo con inserciones manuales
CREATE TABLE log_errores (
    id_log NUMBER GENERATED BY DEFAULT ON NULL AS IDENTITY PRIMARY KEY,
    mensaje VARCHAR2(500)
);
-- Insercion en Caso 2 enviando un NULL (Oracle generara el ID)
INSERT INTO log_errores (id_log, mensaje) VALUES (NULL, 'Error en
   conexion');
```

Consideraciones y Restricciones

- **Unicidad de columna:** Solo puede existir **una sola columna** de identidad por tabla.
- **Tipo de dato:** La columna debe ser de tipo numérico (`NUMBER` o sus alias).
- **Huecos en la secuencia:** Al igual que las secuencias normales, si una transacción falla o se hace `ROLLBACK`, el número generado se pierde, pudiendo aparecer *saltos* en la numeración.

9. DDL III: Vistas (VIEWS)

Una vista es una consulta SQL guardada en el diccionario de datos que se presenta ante el usuario como si fuera una tabla real, pero sin almacenar datos físicamente. Se les conoce como “tablas virtuales”.

SINTAXIS GENÉRICA

```
CREATE [OR REPLACE] VIEW ⟨nombre_vista⟩ AS
  SELECT ⟨columnas⟩
  FROM ⟨tablas⟩
  [WHERE ⟨condiciones⟩];
```

9.1. Utilidad y Ventajas

- **Simplificación:** Oculta la complejidad de los `JOIN` entre varias tablas.
- **Seguridad:** Permite que un usuario vea solo ciertas columnas de una tabla sensible.
- **Consistencia:** Centraliza la lógica de negocio en un solo lugar (como, por ejemplo, mostrar solo proyectos “ACTIVOS”).

Código 25: Ejemplo: Vista Detallada de Proyectos

```
-- Creamos una vista que une proyectos con sus departamentos
CREATE OR REPLACE VIEW vw_resumen_proyectos AS
SELECT p.id, p.titulo, d.nombre AS nombre_departamento, p.estado
    FROM proyecto p
    JOIN departamento d
        ON p.dep_id = d.id_dep;

-- El usuario consulta la vista de forma sencilla
SELECT * FROM vista_resumen_proyectos WHERE estado = 'ACTIVO';
```

10. DDL IV: Sinónimos (SYNONYMS)

Un sinónimo es un nombre alternativo (alias) para un objeto de la base de datos (tablas, vistas, secuencias o procedimientos).

SINTAXIS GENÉRICA

```
CREATE [PUBLIC] SYNONYM ⟨nombre_alias⟩ FOR ⟨objeto_original⟩;
```

10.1. Tipos y Casos de Uso

- **Privados:** Pertenecen a un esquema y solo son visibles por su dueño o quien tenga permisos sobre ellos.
- **Públicos (`PUBLIC`):** Son visibles para todos los usuarios de la base de datos.

¿Por qué usarlos? Principalmente para la **transparencia de ubicación**.
Si el usuario `vendedor` necesita acceder a la tabla `departamento` del esquema `admin_clase` (usuario en Oracle), tendría que escribir `admin_clase.departamento`. Con un sinónimo, solo escribe `departamento`.

Código 26: Ejemplo: Creación de Sinónimo

```
-- Supongamos que estamos logueados como el usuario 'vendedor'
-- Creamos un alias para no tener que usar el prefijo del esquema
CREATE SYNONYM depto FOR admin_clase.departamento;

-- Ahora la consulta es mas limpia
SELECT * FROM depto;
```

10.2. Consulta y Diccionario de Datos

Para saber qué sinónimos existen en la base de datos y a qué objeto real están apuntando, consultamos las vistas del diccionario de datos. Las columnas más importantes son `SYNONYM_NAME` (el alias) y `TABLE_NAME` (el objeto original).

- `USER_SYNONYMS`**:** Sinónimos que pertenecen al usuario actual.
- `ALL_SYNONYMS`**:** Todos los sinónimos a los que el usuario tiene acceso.
- `DBA_SYNONYMS`**:** Todos los sinónimos de la base de datos (requiere privilegios de administrador).

Código 27: Consulta de equivalencias de sinónimos

```
-- Averiguar a que tabla apunta el sinonimo 'depto'
SELECT synonym_name, table_owner, table_name
    FROM user_synonyms
    WHERE synonym_name = 'DEPTO';

-- Ver todos los sinonimos publicos del sistema
SELECT owner, synonym_name, table_name
    FROM all_synonyms
    WHERE owner = 'PUBLIC';
```

10.3. Permisos Necesarios (DCL)

La gestión de sinónimos requiere privilegios específicos de sistema que deben ser otorgados por un administrador (`SYS` o `SYSTEM`).

Privilegio	Descripción Técnica
`CREATE SYNONYM`	Permite crear sinónimos privados en el propio esquema del usuario.
`CREATE PUBLIC SYNONYM`	Permite crear alias visibles para todos los usuarios (uso global).
`CREATE ANY SYNONYM`	Permite crear sinónimos en cualquier esquema de la base de datos.

Cuadro 7: Privilegios para la gestión de Sinónimos

En el siguiente ejemplo se puede ver cómo usarlo de forma práctica:

Código 28: Concesion de privilegios de sinonimos

```
-- Como administrador, permitimos que el usuario cree sus propios alias
GRANT CREATE SYNONYM TO vendedor;

-- Permitimos la creacion de nombres globales (requiere cuidado)
GRANT CREATE PUBLIC SYNONYM TO admin_clase;
```

Nota importante sobre seguridad:
El hecho de tener un sinónimo **no otorga permisos automáticos** sobre el objeto. Si el usuario `vendedor` crea un sinónimo para la tabla `admin_clase.departamento`, seguirá necesitando que el dueño le de un `GRANT SELECT` sobre la tabla original para poder ver los datos. El sinónimo es solo un “puntero” de nombre, no una llave de acceso.

11. DDL V: Modificación y Borrado de Objetos

El diseño de la base de datos raramente es definitivo. A menudo necesitamos alterar la estructura de las tablas existentes.

11.1. Modificación de Estructura (ALTER TABLE)

Podemos añadir columnas nuevas, modificar las existentes o renombrarlas.

SINTAXIS GENÉRICA: ADD Y MODIFY

```
-- Añadir nuevas columnas
ALTER TABLE ⟨nombre_tabla⟩
  ADD ( ⟨columna⟩ ⟨tipo⟩ [DEFAULT ⟨valor⟩] [NOT NULL], ... );

-- Modificar columnas existentes (Tipo, Tamaño o Default)
ALTER TABLE ⟨nombre_tabla⟩
  MODIFY ( ⟨columna⟩ ⟨nuevo_tipo⟩ [DEFAULT ⟨nuevo_valor⟩] [NOT NULL],
     ... );
```

11.1.1. Añadir Columnas (ADD)

```
-- Añadimos una columna para el email del responsable a la tabla
   departamento
ALTER TABLE departamento
    ADD email_responsable VARCHAR2(100);
```

11.1.2. Modificar Columnas (MODIFY)

Podemos cambiar el tipo de dato (si la columna está vacía), el tamaño o el valor por defecto.

```
-- Ampliamos el tamaño del nombre y hacemos que sea obligatorio
ALTER TABLE departamento
    MODIFY nombre VARCHAR2(100) NOT NULL;
```

11.1.3. Renombrar Columnas (RENAME COLUMN)

A veces el nombre de una columna no es apropiado y deseamos cambiarlo sin perder los datos.

SINTAXIS GENÉRICA

```
ALTER TABLE nombre_tabla
    RENAME COLUMN nombre_viejo TO nombre_nuevo;
```

```
-- Corregimos el nombre de la columna fecha de inicio en proyecto
ALTER TABLE proyecto RENAME COLUMN f_inicio TO fecha_comienzo;
```

11.1.4. Eliminar Columnas (DROP vs SET UNUSED)

Eliminar una columna físicamente consume muchos recursos si la tabla es grande.

- **DROP COLUMN:** Borra la columna y los datos inmediatamente. Lento en tablas grandes.
- **SET UNUSED:** Marca la columna como ïnvisibleïnstantáneamente. Los datos siguen en disco pero nadie puede verlos. Luego, en una ventana de mantenimiento, liberamos el espacio.

Código 29: Borrado eficiente de columnas

```
-- Opción A: Borrado inmediato para tablas con pocos datos
ALTER TABLE proyecto DROP COLUMN estado;

-- Opción B: Borrado diferido para optimizar rendimiento
-- Marcamos la columna como no usada instantáneamente
ALTER TABLE proyecto SET UNUSED (fecha_comienzo);

-- Posteriormente liberamos el espacio físico en disco
ALTER TABLE proyecto DROP UNUSED COLUMNS;
```

11.2. Gestión de Restricciones (Enable/Disable)

A veces, para realizar cargas masivas de datos (ETL), nos interesa desactivar las validaciones temporalmente para ganar velocidad.

```
-- Desactivamos la FK para cargar datos sin comprobar el padre
ALTER TABLE proyecto DISABLE CONSTRAINT proyecto_fk_dep_id;

-- Realizamos la carga masiva de datos

-- Reactivamos la restricción para que Oracle valide los datos
ALTER TABLE proyecto ENABLE CONSTRAINT proyecto_fk_dep_id;
```

12. Gestión de la papelera y formas de borrado

12.1. Eliminación de Tablas: DROP vs TRUNCATE

Esta es una distinción fundamental en Oracle Database.

Comando	Tipo	Comportamiento
`DROP TABLE`	DDL	Elimina la estructura de la tabla y sus datos. La tabla desaparece del diccionario. Puede ir a la Papelera de Reciclaje.
`TRUNCATE TABLE`	DDL	Elimina **todas** las filas. Mantiene la estructura. Resetea el espacio y la "marca de agua"(High Water Mark). **No se puede deshacer (ROLLBACK)**. Es extremadamente rápido.
`DELETE FROM`	DML	Borra filas una a una. Genera logs de deshacer (Undo). Es lento para borrados totales. Permite **ROLLBACK**.

Cuadro 8: Diferencias de Borrado

Código 30: Ejemplos de Borrado

```
-- Vaciamos la tabla completamente reseteando su almacenamiento
TRUNCATE TABLE proyecto;

-- Eliminamos la tabla del sistema evitando la papelera de reciclaje
DROP TABLE proyecto PURGE;
```

Borrado de Tablas con Dependencias (Padres): Si intentamos hacer un `DROP TABLE departamento`, Oracle lanzará el error `ORA-02449` si existen claves foráneas apuntando a ella. Para forzar el borrado sin eliminar antes las tablas hijas, usamos:

```
-- Borra la tabla y elimina automáticamente las FKs que la apuntan
DROP TABLE departamento CASCADE CONSTRAINTS;
```

12.2. La Papelera de Reciclaje (Recycle Bin)

La **Recycle Bin** es un contenedor lógico (una tabla del diccionario de datos) donde Oracle almacena los objetos eliminados. En lugar de borrar físicamente los datos, el sistema simplemente renombra el objeto y sus dependencias (índices, disparadores, etc.) con un prefijo único del tipo `BIN$`.

Funcionamiento y Características

- **Espacio:** Los objetos en la papelera siguen ocupando el mismo espacio en su *tablespace* original y cuentan para la cuota del usuario.
- **Borrado Automático:** Si el *tablespace* se queda sin espacio, Oracle eliminará automáticamente los objetos de la papelera empezando por los más antiguos (mecanismo FIFO).
- **Limitaciones:** No se pueden realizar operaciones DML (INSERT/UPDATE) o DDL sobre objetos que están en la papelera; solo se permite su consulta (`SELECT`).

Comandos de Gestión

Código 31: Gestión de la Recycle Bin

```
-- Ver el contenido de la papelera del usuario actual
SHOW RECYCLEBIN;
-- También se puede hacer una consulta detallada:
SELECT original_name, object_name, type, droptime
    FROM user_recyclebin;

-- Recuperar una tabla (Flashback Drop)
FLASHBACK TABLE nombre_tabla
    TO BEFORE DROP;
-- Opcionalmente, podemos renombrarla al recuperarla:
FLASHBACK TABLE proyecto
    TO BEFORE DROP
    RENAME TO proyecto_recuperado;

-- Borrado permanente (Saltarse la papelera)
DROP TABLE empleados PURGE;

-- Limpieza manual (Liberar espacio)
PURGE TABLE empleados;        -- Elimina una tabla específica de la
   papelera
PURGE RECYCLEBIN;             -- Vacía la papelera del usuario actual
PURGE DBA_RECYCLEBIN;         -- Vacía toda la papelera (requiere
   privilegios de DBA)
PURGE TABLESPACE users;       -- Elimina objetos de la papelera de un
   tablespace
```

¿Cómo consultar un objeto borrado?

Si necesitas ver los datos de una tabla antes de recuperarla, puedes hacer un `SELECT` usando su nombre de sistema (entre comillas dobles):

```
SELECT * FROM "BIN$u5X...";
```

13. DCL II: Gestión Avanzada de Permisos

Aunque ya hemos visto los comandos básicos GRANT y REVOKE, la seguridad en Oracle es granular. Es fundamental distinguir entre tener permiso para hacer algo en el sistema (crear una tabla) y tener permiso para manipular los datos de otro usuario.

13.1. Tipos de Privilegios

- **Privilegios de Sistema:** Permiten realizar acciones generales en la base de datos (como CREATE SESSION, CREATE TABLE, CREATE USER, ...). Son otorgados generalmente por el DBA.
- **Privilegios de Objeto:** Permiten realizar acciones sobre un objeto concreto que pertenece a otro usuario (p. ej.: hacer un SELECT sobre la tabla empleados del usuario RRHH).

13.2. La Cláusula WITH GRANT OPTION

Cuando otorgamos un permiso a un usuario, a veces queremos que ese usuario sea capaz de "re-conceder" ese permiso a otros. Esto se controla con cláusulas especiales según el tipo de privilegio.

SINTAXIS GENÉRICA

```
-- Para privilegios de OBJETO
GRANT ⟨permiso⟩ ON ⟨objeto⟩ TO ⟨usuario⟩ [WITH GRANT OPTION];

-- Para privilegios de SISTEMA
GRANT ⟨permiso⟩ TO ⟨usuario⟩ [WITH ADMIN OPTION];
```

Código 32: Ejemplo de Delegación de Permisos

```
-- A. El usuario RRHH da permiso de lectura a JEFE_PROYECTO
-- y ademas le permite que el pueda darselo a sus subordinados.
GRANT SELECT ON empleados TO jefe_proyecto WITH GRANT OPTION;

-- B. Ahora, el usuario JEFE_PROYECTO puede hacer esto:
GRANT SELECT ON rrhh.empleados TO programador_junior;

-- C. Si no tuviera el WITH GRANT OPTION, el paso C daria error.
```

13.3. Tabla de Privilegios de Sistema Esenciales

Para que un desarrollador pueda trabajar con normalidad, no basta con el rol CONNECT. A continuación se detallan los permisos atómicos necesarios para el día a día.

Privilegio	Descripción y Alcance
`CREATE SESSION`	**Vital.** Permite conectar a la base de datos. Sin esto, el usuario no puede hacer login.
`CREATE TABLE`	Permite crear tablas **en su propio esquema**.
`CREATE VIEW`	Permite crear vistas. (Nota: para crear una vista sobre tablas de otro usuario, se requieren permisos de SELECT sobre esas tablas).
`CREATE SEQUENCE`	Permite crear secuencias para autogeneración de claves.
`CREATE SYNONYM`	Permite crear sinónimos privados.
`CREATE PROCEDURE`	Necesario para programar en PL/SQL (procedimientos, funciones y paquetes).
`CREATE TRIGGER`	Permite crear disparadores en sus propias tablas.
`UNLIMITED TABLESPACE`	**Crítico.** Ignora las cuotas de disco. Sin este permiso o una cuota definida, recibiremos el error `ORA-01536` al intentar insertar datos.

Cuadro 9: Privilegios de Sistema Comunes

13.4. Roles Predefinidos: CONNECT y RESOURCE

Para simplificar la administración, Oracle agrupa estos permisos en **Roles**. En entornos de desarrollo rápido o clases, es común ejecutar:

```
GRANT CONNECT, RESOURCE TO usuario;
```

- **CONNECT:** En versiones modernas (12c+), básicamente solo otorga `CREATE SESSION`.
- **RESOURCE:** Es una colección heredada que incluye `CREATE TABLE`, `SEQUENCE`, `TRIGGER`, `PROCEDURE`, etc. **Nota:** Curiosamente, el rol RESOURCE **no** incluye `CREATE VIEW`, por lo que a menudo hay que darlo por separado.

13.5. Revocación y Efecto Cascada

Cuando usamos `REVOKE`, el comportamiento depende de cómo se otorgó el permiso.

- **Sin cascada (Sistema):** Si A da permiso a B (con admin option), y B da a C. Si A revoca a B, C **mantiene** el permiso.
- **Con cascada (Objetos):** Si A da permiso a B (con grant option), y B da a C. Si A revoca a B, **Oracle revoca automáticamente el permiso a C**.

Código 33: Revocar permisos

```
-- Quitar permiso de lectura y detener la cadena de confianza
REVOKE SELECT ON empleados FROM jefe_proyecto;
-- Esto revocara automaticamente el permiso al 'programador_junior'
-- del ejemplo anterior.
```

14. Bibliografía y Recursos Recomendados

Para profundizar en los temas tratados, se recomienda consultar las siguientes fuentes oficiales y de la comunidad.

- **Oracle Database Documentation:**
  La fuente oficial de referencia para sintaxis y arquitectura.
  https://docs.oracle.com/en/database/oracle/oracle-database/
- **Oracle Base (Tim Hall):**
  Excelente recurso con guías paso a paso sobre características específicas y novedades.
  https://oracle-base.com/
- **Oracle Live SQL:**
  Entorno gratuito en la nube para probar scripts SQL sin instalar nada.
  https://livesql.oracle.com/
- **Ask TOM:**
  Repositorio de preguntas y respuestas técnicas resueltas por expertos de Oracle.
  https://asktom.oracle.com/